Library Management System

Using Binary Search Tree (BST)

Course: BACSE105 Data Structures and Algorithms (WINTER SEMESTER 2025-2026)

Names & Registration Numbers:
Rajvardhan Jain (25BCE2539)
Aviral Srivastava (25BCE2019)
Abhay Kumar (25BCE2537)

Course Instructor: Dr. Antony Seba P

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

Date of Submission: 17th April 2026

1. Problem Statement

A library maintains a collection of books that grows over time. The system must store book records in a sorted manner based on ISBN or title to enable quick searching, insertion, and deletion. An efficient approach is required to manage the collection dynamically without slowing down operations as the number of books increases.

Traditional array-based or linked-list approaches suffer from either O(n) search time or costly re-sorting upon every insertion. A dynamic, self-ordering data structure is needed that keeps records sorted at all times and supports O(log n) average-case performance for all core operations.

2. Objective

The objectives of this project are:
• Design and implement a Library Management System using a Binary Search Tree (BST) keyed on ISBN or book title.
• Support efficient insertion of new book records while maintaining sorted order automatically.
• Provide O(log n) average-case search by ISBN or title for quick lookup.
• Enable deletion of book records, handling leaf, one-child, and two-child cases correctly.
• Perform in-order traversal to display all books in alphabetically or numerically sorted order.
• Analyze and compare BST performance against a naive unsorted list and quantify the improvement.

3. Data — Fields Used to Build the Solution

Each book node stored in the BST contains the following fields:

Field	Data Type	Description
isbn	string	13-digit International Standard Book Number (primary key)
title	string	Title of the book (alternate key for title-based BST)
author	string	Name of the author(s)
genre	string	Genre/category (e.g., Fiction, Science, History)
year	int	Year of publication
copies	int	Number of copies available in the library
isAvailable	bool	Whether at least one copy is currently available

4. Data Structures Used

4.1 Binary Search Tree (BST)

The primary data structure is a Binary Search Tree. Each node holds a Book record and two pointers: left (smaller key) and right (larger key). The BST invariant ensures all records are kept in sorted order implicitly.

Node structure (C++):

```
struct Book {
string isbn, title, author, genre;
int year, copies;
bool isAvailable;
};
struct BookNode {
Book data; // The book record
BookNode* left; // Left child (smaller key)
BookNode* right; // Right child (larger key)
BookNode(Book b) : data(b), left(nullptr), right(nullptr) {}
};
```

4.2 Why BST over Alternatives

Data Structure	Search	Insert	Delete	Sorted Order
Unsorted Array	O(n)	O(1)	O(n)	No
Sorted Array	O(log n)	O(n)	O(n)	Yes
Linked List	O(n)	O(1)	O(n)	No
BST (average)	O(log n)	O(log n)	O(log n)	Yes (in-order)
Balanced BST/AVL	O(log n)	O(log n)	O(log n)	Yes (guaranteed)

5. Algorithm Design

5.1 Insertion

Compare the new book's key with the current node's key. Traverse left if smaller, right if greater. Insert at the first NULL position encountered.

```
insert(root, book):
if root == NULL: return new node(book)
if book.isbn < root.isbn:
root.left = insert(root.left, book)
else if book.isbn > root.isbn:
root.right = insert(root.right, book)
return root
```

5.2 Search

Compare the target key at each node. Go left if smaller, right if greater, return node if equal.

```
search(root, isbn):
if root == NULL or root.isbn == isbn: return root
if isbn < root.isbn: return search(root.left, isbn)
return search(root.right, isbn)
```

5.3 Deletion

Three cases:

(1) Leaf node — simply remove.

(2) One child — replace node with child.

(3) Two children — replace node's data with in-order successor, then delete the successor.

```
delete(root, isbn):
if root == NULL: return NULL
if isbn < root.isbn: root.left = delete(root.left, isbn)
else if isbn > root.isbn: root.right = delete(root.right, isbn)
else:
if root.left == NULL: return root.right
if root.right == NULL: return root.left
successor = findMin(root.right)
root.data = successor.data
root.right = delete(root.right, successor.isbn)
return root
```

5.4 In-Order Traversal (Sorted Display)

```
inorder(root):
if root == NULL: return
inorder(root.left)
print(root.data) // visit
inorder(root.right)
```

6. Optimization Techniques

6.1 Improvements over the Naive Approach

• Naive unsorted list requires O(n) linear search for every lookup; BST reduces this to O(log n) by leveraging sorted tree structure.

• Sorted array insertion is O(n) due to element shifting; BST insertion is O(log n) via pointer manipulation only.

• Dual BST indexing (by ISBN and title) allows O(log n) lookup on either key without maintaining two full sorted arrays.

6.2 Space / Time Trade-offs

Maintaining two BSTs doubles memory for node pointers but provides O(log n) access on both keys simultaneously. Compared to keeping two sorted arrays and re-sorting on each insertion, the BST approach saves O(n) time per insertion.

6.3 Future Optimization: AVL / Red-Black Tree

The standard BST degrades to O(n) in the worst case when books are inserted in sorted ISBN order (degenerate linear chain). An AVL Tree or Red-Black Tree auto-balances after every insertion and deletion, guaranteeing O(log n) performance in all cases. This is identified as the production-grade upgrade path.

7. Implementation

Language: C++ | **Platform:** Linux / Windows

```
#include <iostream>
#include <string>
using namespace std;
struct Book {
string isbn, title, author, genre;
int year, copies; bool isAvailable;
};
struct Node {
Book data; Node *left, *right;
Node(Book b) : data(b), left(nullptr), right(nullptr) {}
};

class LibraryBST {
Node* root;
Node* insert(Node* r, Book b) {
if (!r) return new Node(b);
if (b.isbn < r->data.isbn) r->left = insert(r->left, b);
else if (b.isbn > r->data.isbn) r->right = insert(r->right, b);
return r;
}
Node* search(Node* r, string isbn) {
if (!r || r->data.isbn == isbn) return r;
return isbn < r->data.isbn ? search(r->left,isbn) : search(r->right,isbn);
}
Node* minNode(Node* r) { while(r->left) r=r->left; return r; }
Node* del(Node* r, string isbn) {
if (!r) return r;
if (isbn < r->data.isbn) r->left = del(r->left, isbn);
else if (isbn > r->data.isbn) r->right = del(r->right, isbn);
else {
if (!r->left) return r->right;
```

```
if (!r->right) return r->left;
Node* s = minNode(r->right);
r->data = s->data;
r->right = del(r->right, s->data.isbn);
}
return r;
}
void inorder(Node* r) {
if (!r) return;
inorder(r->left);
cout << r->data.isbn << " | " << r->data.title
<< " | " << r->data.copies << endl;
inorder(r->right);
}
public:
LibraryBST() : root(nullptr) {}
void insert(Book b) { root = insert(root, b); }
Node* search(string isbn) { return search(root, isbn); }
void remove(string isbn) { root = del(root, isbn); }
void display() { inorder(root); }
};
```

8. Sample Input and Output (Minimum 5 Sets)

Set 1 — Insert books & display sorted by ISBN

```
Input: Insert books with ISBNs: 9780131103627, 9780201633610,
9780596517748, 9780132350884, 9780201485677

Output (In-Order):
9780131103627 | The C Programming Language | K&R; | 3 copies
9780132350884 | Clean Code | R. Martin | 2 copies
9780201485677 | The Pragmatic Programmer | Hunt&Thomas; | 4 copies
9780201633610 | Design Patterns | GoF | 1 copy
9780596517748 | JavaScript: The Good Parts | D. Crockford| 5 copies
```

Set 2 — Search by ISBN (found)

```
Input: search("9780201633610")

Output: FOUND -> 9780201633610 | Design Patterns | GoF | SE | 1 copy
```

Set 3 — Search by ISBN (not found)

```
Input: search("9999999999999")
Output: Book not found.
```

Set 4 — Delete a leaf node

```
Input: remove("9780596517748") // leaf node
display()

Output (In-Order after deletion):
9780131103627 | The C Programming Language | K&R; | 3 copies
9780132350884 | Clean Code | R. Martin | 2 copies
9780201485677 | The Pragmatic Programmer | Hunt&Thomas; | 4 copies
9780201633610 | Design Patterns | GoF | 1 copy
```

Set 5 — Delete a node with two children

```
Input: remove("9780201485677") // two-child node
display()

Output: Replaced with in-order successor (9780201633610).
9780131103627 | The C Programming Language | K&R; | 3 copies
9780132350884 | Clean Code | R. Martin | 2 copies
9780201633610 | Design Patterns | GoF | 1 copy
```

Set 6 — Insert after deletion (dynamic growth)

```
Input: insert({"9780134685991", "Effective Java", "J. Bloch", ...})
display()

Output:
9780131103627 | The C Programming Language | K&R; | 3 copies
9780132350884 | Clean Code | R. Martin | 2 copies
9780134685991 | Effective Java | J. Bloch | 6 copies
9780201633610 | Design Patterns | GoF | 1 copy
```

9. Time and Space Complexity Analysis

9.1 Time Complexity

Operation	Best Case	Average Case	Worst Case (Skewed Tree)
Insert	O(log n)	O(log n)	O(n)
Search	O(1)	O(log n)	O(n)
Delete	O(log n)	O(log n)	O(n)
In-Order	O(n)	O(n)	O(n)
Find Minimum	O(log n)	O(log n)	O(n)

Worst-case O(n) occurs only when books are inserted in already-sorted ISBN order, producing a degenerate linear chain. This is mitigated in production by AVL or Red-Black trees.

9.2 Space Complexity

Component	Space
BST node storage	O(n) — one node per book
Recursion stack (search/insert/del)	O(h) — h = height of tree
Average height h	O(log n) for balanced input
Worst-case height h	O(n) for sorted-order input
Dual BST (ISBN + title index)	O(2n) = O(n) overall

10. Conclusion

This project successfully demonstrates the application of Binary Search Trees to the real-world problem of managing a dynamically growing library collection. By organising book records in a BST keyed on ISBN (or title), the system achieves O(log n) average-case performance for insertion, deletion, and search — a significant improvement over naive O(n) linear structures.

• BST maintains implicit sorted order, enabling in-order traversal to list books alphabetically or numerically without any additional sorting step.

• Dual BST indexing provides fast multi-key lookups at the cost of only doubled pointer storage.

• The deletion algorithm correctly handles all three structural cases (leaf, one child, two children) using the in-order successor strategy.

• Performance degrades to O(n) for sorted insertions, highlighting the value of balanced variants (AVL, Red-Black) for production systems.

Overall, the project solidifies understanding of recursive tree algorithms, pointer-based dynamic data structures, and the practical trade-offs between time complexity, space complexity, and implementation simplicity.

11. GitHub Link

Repository: https://github.com/rajvardhanjainsoni/DSA-Project-Library-System

12. References

• Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

• Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in C++* (4th ed.). Pearson.

• GeeksforGeeks. Binary Search Tree – Insert, Delete, Search. https://www.geeksforgeeks.org/binary-search-tree-data-structure/

• Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

• cppreference.com. std::map and std::set (BST-based containers). https://en.cppreference.com